DS H.W SET 7

1) Write a C program to construct an expression tree for a given expression.

eg. (a - b) + (c * d)

(+)

**/ **

(-) (*)

**/ \ / **

a b c d

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#define MAX 100
typedef struct Node {
char data;
struct Node *left, *right;
} Node;
 Node* nodeStack[MAX];
int topNode = -1;
 char opStack[MAX];
int topOp = -1;
 Node* createNode(char data) {
Node* newNode = (Node*)malloc(sizeof(Node));
newNode->data = data;
newNode->left = newNode->right = NULL;
return newNode;
}
 void pushNode(Node* node) {
nodeStack[++topNode] = node;
}
 Node* popNode() {
return nodeStack[topNode--];
}
```

```
void pushOp(char op) {
opStack[++topOp] = op;
}
 char popOp() {
return opStack[topOp--];
}
 int precedence(char op) {
if(op == '+' || op == '-') return 1;
if(op == '*' || op == '/') return 2;
return 0;
}
 void buildTree() {
char op = popOp();
Node* right = popNode();
Node* left = popNode();
Node* newNode = createNode(op);
newNode->left = left;
newNode->right = right;
pushNode(newNode);
}
Node* constructTree(char infix[]) {
int i;
for(i = 0; infix[i] != '\0'; i++) {
char ch = infix[i];
if(ch == ' ')
continue;
if(isalnum(ch)) {
pushNode(createNode(ch));
}
else if(ch == '(') {
pushOp(ch);
}
```

```
else if(ch == ')') {
while(topOp != -1 && opStack[topOp] != '('
buildTree();
popOp();
}
else {
while(topOp != -1 &&
precedence(opStack[topOp]) >= precedence(ch))
buildTree();
pushOp(ch);
}
}
while(topOp != -1)
buildTree();
return popNode();
}
void inorder(Node* root) {
if(root) {
inorder(root->left);
printf("%c ", root->data);
inorder(root->right);
}
}
void printTree(Node* root, int level) {
if(root == NULL)
return;
printTree(root->right, level + 1);
for(int i = 0; i < level; i++)
printf(" ");
printf("%c\n", root->data);
printTree(root->left, level + 1);
}
```

```
int main() {
char expression[MAX];
printf("Enter an infix expression: ");
scanf("%s", expression);
Node* root = constructTree(expression);
printf("\nInorder Traversal of Expression Tree:\n");
inorder(root);
printf("\n\nExpression Tree Structure:\n\n");
printTree(root, 0);
return 0;
}
```

Output

```
Enter an infix expression: (a-b)+(c*d)
Inorder Traversal of Expression Tree:
a - b + c * d
```

2) Write a C program to check whether the given binary tree is a binary search tree.

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
 struct Node {
int data;
struct Node *left, *right;
};
 struct Node* createNode(int data) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = data;
newNode->left = newNode->right = NULL;
return newNode;
}
 struct Node* buildTree() {
int value;
printf("Enter node value (-1 for no node): ");
scanf("%d", &value);
if (value == -1)
return NULL;
struct Node* root = createNode(value);
printf("Enter left child of %d\n", value);
root->left = buildTree();
printf("Enter right child of %d\n", value);
root->right = buildTree();
return root;
}
int isBSTUtil(struct Node* root, int min, int max) {
if (root == NULL)
return 1;
if (root->data <= min || root->data >= max)
return 0;
```

```
return isBSTUtil(root->left, min, root->data) &&
isBSTUtil(root->right, root->data, max);
}
int isBST(struct Node* root) {
return isBSTUtil(root, INT_MIN, INT_MAX);
}
void inorder(struct Node* root) {
if (root != NULL) {
inorder(root->left);
printf("%d ", root->data);
inorder(root->right);
}
}
int main() {
struct Node* root;
printf("Create Binary Tree\n");
printf("Use -1 for NULL node\n\n");
root = buildTree();
printf("\nInorder Traversal: ");
inorder(root);
printf("\n");
if (isBST(root))
printf("The given Binary Tree is a Binary Search Tree (BST).\n");
else
printf("The given Binary Tree is NOT a Binary Search Tree (BST).\n");
return 0;
}
```

Output

```
Create Binary Tree
Use -1 for NULL node
Enter node value (-1 for no node): 50
Enter left child of 50
Enter node value (-1 for no node): 30
```

Enter left child of 30

Enter node value (-1 for no node): 20

Enter left child of 20

Enter node value (-1 for no node): -1

Enter right child of 20

Enter node value (-1 for no node): -1

Enter right child of 30

Enter node value (-1 for no node): 40

Enter left child of 40

Enter node value (-1 for no node): -1

Enter right child of 40

Enter node value (-1 for no node): -1

Enter right child of 50

Enter node value (-1 for no node): 70

Enter left child of 70

Enter node value (-1 for no node): 60

Enter node value (-1 for no node): -1

Enter node value (-1 for no node): -1

Enter right child of 70

Enter node value (-1 for no node): 80

Enter node value (-1 for no node): -1

Enter node value (-1 for no node): -1

Inorder Traversal: 20 30 40 50 60 70 80

The given Binary Tree is a Binary Search Tree (BST).

3) Write a C program to implement a max heap.

```
#include <stdio.h>
#define MAX 100
void swap(int *a, int *b)
{
int temp = *a;
*a = *b;
*b = temp;
}
void heapify(int heap[], int n, int i)
{
int largest = i;
int left = 2 * i + 1;
int right = 2 * i + 2;
if (left < n && heap[left] > heap[largest])
largest = left;
if (right < n && heap[right] > heap[largest])
largest = right;
if (largest != i)
{
swap(&heap[i], &heap[largest]);
heapify(heap, n, largest);
}
}
void buildMaxHeap(int heap[], int n)
{
for (int i = n / 2 - 1; i >= 0; i--)
{
heapify(heap, n, i);
}
}
void displayHeap(int heap[], int n)
{
```

```
printf("\nMax Heap Elements:\n");
for (int i = 0; i < n; i++)
{
printf("%d ", heap[i]);
}
printf("\n");
}
int main()
{
int heap[MAX];
int n;
printf("Enter number of elements: ");
scanf("%d", &n);
printf("Enter %d elements:\n", n);
for (int i = 0; i < n; i++)
{
scanf("%d", &heap[i]);
}
buildMaxHeap(heap, n);
displayHeap(heap, n);
printf("\nMaximum element in the heap = %d\n", heap[0]);
return 0;
}
```

Output

```
Enter number of elements: 7
Enter 7 elements:
10 20 15 30 40 50 60
```

4) Given an undirected graph, the task is to check if there is a cycle in the given graph.

```
#include <stdio.h>
 #define MAX 100
int graph[MAX][MAX];
int visited[MAX];
int vertices;
int dfs(int current, int parent)
{
visited[current] = 1;
for (int i = 0; i < vertices; i++)
{
if (graph[current][i])
{
if (!visited[i])
{
if (dfs(i, current))
return 1;
}
else if (i != parent)
{
return 1;
}
}
}
return 0;
}
int containsCycle()
{
for (int i = 0; i < vertices; i++)
{
visited[i] = 0;
}
```

```
for (int i = 0; i < vertices; i++)
{
if (!visited[i])
{
if (dfs(i, -1))
return 1;
}
}
return 0;
}
int main()
{
int edges;
int u, v;
printf("Enter number of vertices: ");
scanf("%d", &vertices);
printf("Enter number of edges: ");
scanf("%d", &edges);
// Initialize graph
for (int i = 0; i < vertices; i++)
{
for (int j = 0; j < vertices; j++)
{
graph[i][j] = 0;
}
}
printf("Enter the edges (u v):\n");
for (int i = 0; i < edges; i++)
{
scanf("%d %d", &u, &v);
graph[u][v] = 1;
graph[v][u] = 1; // Undirected graph
}
```

```
if (containsCycle())
printf("\nThe graph contains a cycle.\n");
else
printf("\nThe graph does NOT contain a cycle.\n");
return 0;
}
```

Output

```
Enter number of vertices: 4
Enter number of edges: 4
Enter the edges (u v):
0 1
1 2
2 3
3 0
The graph contains a cycle.
```

5) Given an n x m grid of 'W' (Water) and 'L' (Land). The task is to count the number of islands.

I/P: grid[][] = {

{'L', 'L', 'W', 'W', 'W'},

{'L', 'L', 'W', 'W', 'L'},

{'W', 'W', 'L', 'W', 'W'},

{'W', 'W', 'W', 'W', 'W'},

{'L', 'L', 'W', 'L', 'L'}

}

```
#include <stdio.h>
#define MAX 100
char grid[MAX][MAX];
int visited[MAX][MAX];
int rows, cols;
int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};
void dfs(int x, int y)
{
visited[x][y] = 1;
for (int i = 0; i < 4; i++)
{
int newX = x + dx[i];
int newY = y + dy[i];
if (newX >= 0 && newX < rows &&
newY >= 0 && newY < cols &&
!visited[newX][newY] &&
grid[newX][newY] == 'L')
{
dfs(newX, newY);
}
}
}
int countIslands()
{
```

```
int count = 0;
for (int i = 0; i < rows; i++)
{
for (int j = 0; j < cols; j++)
{
if (grid[i][j] == 'L' && !visited[i][j])
{
dfs(i, j);
count++;
}
}
}
return count;
}
int main()
{
printf("Enter number of rows: ");
scanf("%d", &rows);
printf("Enter number of columns: ");
scanf("%d", &cols);
printf("\nEnter the grid (L for Land, W for Water):\n");
for (int i = 0; i < rows; i++)
{
for (int j = 0; j < cols; j++)
{
scanf(" %c", &grid[i][j]);
visited[i][j] = 0;
}
}
int islands = countIslands();
printf("\nNumber of Islands = %d\n", islands);
return 0;
}
```

Output

Enter number of rows: 5

Enter number of columns: 5

Enter the grid (L for Land, W for Water):

L L W W W

L L W W L

W W L W W

W W W W W

L L W L L

6) Find out the minimum spanning tree using Kruskal's algorithm.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
struct Edge {
int src, dest, weight;
};
int parent[MAX];
int find(int i)
{
while (parent[i] != i)
i = parent[i];
return i;
}
void Union(int a, int b)
{
int rootA = find(a);
int rootB = find(b);
parent[rootA] = rootB;
}
void sortEdges(struct Edge edges[], int e)
{
struct Edge temp;
for (int i = 0; i < e - 1; i++)
{
for (int j = 0; j < e - i - 1; j++)
{
if (edges[j].weight > edges[j + 1].weight)
{
temp = edges[j];
edges[j] = edges[j + 1];
edges[j + 1] = temp;
}
```

```
}
}
}
int main()
{
int V, E;
printf("Enter number of vertices: ");
scanf("%d", &V);
printf("Enter number of edges: ");
scanf("%d", &E);
struct Edge edges[E];
printf("\nEnter source destination and weight:\n");
for (int i = 0; i < E; i++)
{
scanf("%d %d %d",
&edges[i].src,
&edges[i].dest,
&edges[i].weight);
}
for (int i = 0; i < V; i++)
{
parent[i] = i;
}
sortEdges(edges, E);
printf("\nEdges in Minimum Spanning Tree:\n");
int edgeCount = 0;
int minCost = 0;
for (int i = 0; i < E && edgeCount < V - 1; i++)
{
int srcParent = find(edges[i].src);
int destParent = find(edges[i].dest);
if (srcParent != destParent)
{
```

```
printf("%d -- %d Weight = %d\n",
edges[i].src,
edges[i].dest,
edges[i].weight);
minCost += edges[i].weight;
Union(srcParent, destParent);
edgeCount++;
}
}
printf("\nTotal Cost of MST = %d\n", minCost);
return 0;
}
```

Output

```
Enter number of vertices: 4
Enter number of edges: 5
Enter source destination and weight:
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Edges in Minimum Spanning Tree:
2 -- 3  Weight = 4
0 -- 3  Weight = 5
0 -- 1  Weight = 10
Total Cost of MST = 19
```

7) Use Dijkstra's algorithm to find shortest path from a source to all nodes.

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int minDistance(int dist[], int visited[], int n)
{
int min = INT_MAX, min_index;

for (int v = 0; v < n; v++)
{
if (visited[v] == 0 && dist[v] < min)
{
min = dist[v];
min_index = v;
}
}

return min_index;
}

void dijkstra(int graph[MAX][MAX], int n, int source)
{
int dist[MAX];
int visited[MAX];

for (int i = 0; i < n; i++)
{
dist[i] = INT_MAX;
visited[i] = 0;
}

dist[source] = 0;

for (int count = 0; count < n - 1; count++)
{
int u = minDistance(dist, visited, n);

visited[u] = 1;

for (int v = 0; v < n; v++)
{
if (!visited[v] &&
graph[u][v] &&
dist[u] != INT_MAX &&
dist[u] + graph[u][v] < dist[v])
{
dist[v] = dist[u] + graph[u][v];
}
```

```
}
}

printf("\nShortest distances from Source Vertex %d:\n", source);
printf("Vertex\tDistance\n");

for (int i = 0; i < n; i++)
{
printf("%d\t%d\n", i, dist[i]);
}
}

int main()
{
int n, source;
int graph[MAX][MAX];

printf("Enter the number of vertices: ");
scanf("%d", &n);

printf("\nEnter the adjacency matrix:\n");
printf("(Enter 0 if there is no direct edge)\n");

for (int i = 0; i < n; i++)
{
for (int j = 0; j < n; j++)
{
scanf("%d", &graph[i][j]);
}
}

printf("\nEnter the source vertex: ");
scanf("%d", &source);

dijkstra(graph, n, source);

return 0;
}
```

Output

```
Enter the number of vertices: 5
Enter the adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter the source vertex: 0

Shortest distances from Source Vertex 0:
```

Vertex	Distance
0	0
1	10
2	50
3	30
4	60

8) Write a program to find a mother vertex in the graph. A mother vertex in a graph $G = (V, E)$ is a vertex v such that all other vertices in G can be reached by a path from v.

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];

int visited[MAX];

int V;

void DFS(int v)

{

visited[v] = 1;

for (int i = 0; i < V; i++)

{

if (graph[v][i] == 1 && !visited[i])

{

DFS(i);

}

}

}

int findMotherVertex()

{

int candidate = -1;

for (int i = 0; i < V; i++)

{

visited[i] = 0;

}

for (int i = 0; i < V; i++)

{

if (!visited[i])

{

DFS(i);

candidate = i;

}

}
```

```
for (int i = 0; i < V; i++)
{
visited[i] = 0;
}
DFS(candidate);
for (int i = 0; i < V; i++)
{
if (!visited[i])
return -1;
}
return candidate;
}
int main()
{
int E;
int u, v;
printf("Enter number of vertices: ");
scanf("%d", &V);
printf("Enter number of edges: ");
scanf("%d", &E);
for (int i = 0; i < V; i++)
{
for (int j = 0; j < V; j++)
{
graph[i][j] = 0;
}
}
printf("Enter directed edges (source destination):\n");
for (int i = 0; i < E; i++)
{
scanf("%d %d", &u, &v);
graph[u][v] = 1;
}
```

```
int motherVertex = findMotherVertex();

if (motherVertex == -1)

printf("\nNo Mother Vertex exists in the graph.\n");

else

printf("\nMother Vertex = %d\n", motherVertex);

return 0;

}
```

Output

```
Enter number of vertices: 7

Enter number of edges: 8

Enter directed edges (source destination):

0 1

0 2

1 3

4 1

6 4

5 6

5 2

6 0

Mother Vertex = 5
```

9) A directed graph is strongly connected if there is a path between all pairs of vertices. A strongly connected component of a directed graph is a maximal strongly connected subgraph. Given a graph, print all SCC.

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];

int transpose[MAX][MAX];

int visited[MAX];

int stack[MAX];

int top = -1;

int V;

void push(int v)

{

stack[++top] = v;

}

void DFS1(int v)

{

visited[v] = 1;

for (int i = 0; i < V; i++)

{

if (graph[v][i] && !visited[i])

DFS1(i);

}

push(v);

}

void DFS2(int v)

{

visited[v] = 1;

printf("%d ", v);

for (int i = 0; i < V; i++)

{

if (transpose[v][i] && !visited[i])

DFS2(i);

}
```

```
}
void transposeGraph()
{
for (int i = 0; i < V; i++)
{
for (int j = 0; j < V; j++)
{
transpose[i][j] = graph[j][i];
}
}
}
}
int main()
{
int E;
int u, v;
printf("Enter number of vertices: ");
scanf("%d", &V);
printf("Enter number of edges: ");
scanf("%d", &E);
for (int i = 0; i < V; i++)
{
for (int j = 0; j < V; j++)
{
graph[i][j] = 0;
}
}
printf("Enter directed edges (source destination):\n");
for (int i = 0; i < E; i++)
{
scanf("%d %d", &u, &v);
graph[u][v] = 1;
}
```

```
for (int i = 0; i < V; i++)
visited[i] = 0;
for (int i = 0; i < V; i++)
{
if (!visited[i])
DFS1(i);
}
transposeGraph();
for (int i = 0; i < V; i++)
visited[i] = 0;
printf("\nStrongly Connected Components are:\n");
while (top >= 0)
{
int vertex = stack[top--];
if (!visited[vertex])
{
DFS2(vertex);
printf("\n");
}
}
return 0;
}
```

Output

```
Enter number of vertices: 5
Enter number of edges: 5
Enter directed edges (source destination):
1 0
0 2
2 1
0 3
3 4
Strongly Connected Components are:
0 1 2
```

3

4

10) A bipartite graph G = (V, E) is a graph such that V can be partitioned into two subsets V_1 and V_2 and no edge has both its vertices in the same subset. Write a program to determine whether a graph is a bipartite graph.

Code

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int color[MAX];
int queue[MAX];

int isBipartite(int vertices)
{
int front, rear;

for(int i = 0; i < vertices; i++)
color[i] = -1;

for(int start = 0; start < vertices; start++)
{
if(color[start] != -1)
continue;

front = rear = 0;

queue[rear++] = start;
color[start] = 0;

while(front < rear)
{
int u = queue[front++];

for(int v = 0; v < vertices; v++)
{
if(graph[u][v])
{
// If vertex is uncolored
if(color[v] == -1)
{
color[v] = 1 - color[u];
queue[rear++] = v;
}
// Adjacent vertices have same color
else if(color[v] == color[u])
{
return 0;
}
}
}
```

```
}
}

return 1;
}

int main()
{
int vertices, edges;
int u, v;

printf("Enter number of vertices: ");
scanf("%d", &vertices);

printf("Enter number of edges: ");
scanf("%d", &edges);
for(int i = 0; i < vertices; i++)
{
for(int j = 0; j < vertices; j++)
{
graph[i][j] = 0;
}
}
printf("Enter the edges (u v):\n");

for(int i = 0; i < edges; i++)
{
scanf("%d %d", &u, &v);

graph[u][v] = 1;
graph[v][u] = 1; // Undirected graph
}
if(isBipartite(vertices))
{
printf("\nThe graph is Bipartite.\n");

printf("\nSet V1 : ");
for(int i = 0; i < vertices; i++)
{
if(color[i] == 0)
printf("%d ", i);
}

printf("\nSet V2 : ");
for(int i = 0; i < vertices; i++)
{
if(color[i] == 1)
printf("%d ", i);
}
}
else
```

```
{
printf("\nThe graph is NOT Bipartite.\n");
}

return 0;
}
```

Output

```
Enter number of vertices: 4

Enter number of edges: 4
Enter the edges (u v):
0 1
0 3
2 1
2 3
The graph is Bipartite.
Set V1 : 0 2
Set V2 : 1 3
```